PatientZero

Team workflow and implementation guide

Zero pressure, patient practice.

Current demo: `https://patientzero.me`. This guide describes the current code and the restored EC2 deployment as of 11 October 2026. The demo database now runs PostgreSQL 16 on EC2. The Aurora gateway outage is bypassed; its original database is preserved. Private server configuration stays outside Git.

1 What the product does

PatientZero lets faculty create and approve synthetic patient cases, lets medical students practice encounters, and combines automated evidence with faculty assessment. Conversations remain tied to the case version used when the encounter began.

Workflow to describe during the presentation

- Sign up or sign in as a medical student or faculty member.
- Faculty build a patient manually or generate a draft with AI, then review facts, documents and the rubric.
- Faculty approve the patient; the published version becomes available in the practice library.
- Students conduct chat, voice or video encounters and discover case information through the conversation.
- Students finish the encounter. Faculty review the evidence and release the assessment.
- Students revisit their own history; faculty review encounters, metrics and reported issues across students.

Architecture and where data lives

Component	Current responsibility
Browser	HTML/CSS/JavaScript interface; Three.js avatar; camera preview; speech recognition and audio capture.
Python backend	JSON API, authorization, case validation, dialogue, scoring, storage and background audio jobs.
PostgreSQL on EC2	Users, student/faculty profiles, sessions, OAuth state, approved patient versions and speech-clip metadata.
SQLite on EC2	Working cases/drafts, encounters and text transcripts, reviews, feedback, reply cache, attachment copies and audio outbox.
Private S3	Published patient documents and optional raw audio clips. No student video is stored.
Bedrock / Polly	Sonnet 5 generates patients and replies; Stability generates portraits; Polly supplies speech with a browser fallback.

Team distinction: The app uses both PostgreSQL and SQLite. Text transcripts are in the EC2 workspace database, not loaded from S3. S3 holds binary files and optional audio.

2 Accounts, roles and navigation

Workflow step 1: enter the right workspace.

Features

- Email signup collects name, email, password and role: medical student or faculty. Google sign-in/signup is also available.
- Faculty registration grants faculty access immediately. It does not verify institutional faculty status.
- Login/signup pages use a top brand header. Authenticated pages use the wider sidebar, with the tagline split into two clean lines.
- Both the top profile menu and lower sidebar account card show identity, role and logout. The Link Google buttons have been removed.
- Sessions persist across browser sessions and application restarts. Logout revokes the server-side session.

Tools and implementation

- Five PostgreSQL account tables store users, medical-student profiles, faculty profiles, sessions and OAuth state.
- Passwords use PBKDF2-SHA256 with a random salt and 600,000 iterations. Signup requires an 8–128 character password.
- Random session tokens use 30-day cookies; PostgreSQL stores token hashes. Cookies are HttpOnly, Secure on HTTPS and use SameSite protections. Requests also receive origin/CSRF checks.
- Google uses a server-side authorization-code flow, PKCE, nonce and one-use state. ID-token signature, issuer, audience and expiry are checked.
- Production callback: `https://patientzero.me/api/auth/google/callback`. Client secrets stay in private environment files.
- Server-side role and encounter-owner checks enforce access. Hiding navigation is only the user-interface layer of this protection.

What each role sees

- **Students:** practice library, their encounters/history and allowed assessment views. Other students' encounters are inaccessible.
- **Faculty:** Patient Studio, patient generation, student metrics, all encounter history, rubrics, feedback and source materials.
- **Developer tools:** faculty access plus a separate unlock for diagnostic/export tools; not a student feature.

Presentation answer: "Permissions are enforced by the backend, and students only see their own encounter history. Faculty can review all students. The demo allows users to choose Faculty directly; a production institution would need a faculty-verification policy."

3 Faculty creation and AI generation

Workflow step 2: build a draft, then review it.

Manual creation and supporting materials

- Patient Studio supports demographics, presenting concern, clinical facts, vitals, objectives, behavior and disclosure rules.
- Facts can be available initially, disclosed after an appropriate question, or reserved for faculty. Keywords support progressive disclosure.
- Faculty can add reports and source documents, edit profiles, validate cases, publish them and delete profiles. Existing encounter snapshots survive profile deletion.
- The Generate patient with AI shortcut sits beside Create patient. A creation timestamp helps distinguish generated drafts in the patient list.

AI generation

- Faculty supply a clinical brief and optionally up to four PDF, PNG, JPG, TXT or MD files, each up to 5 MB.
- Temperament and openness sliders control the behavior request. Endpoint legends and the current description have separate aligned positions.
- Bedrock `us.anthropic.claude-sonnet-5` creates the structured patient draft and rubric. A global Sonnet 5 profile is a configured fallback.
- The backend uses structured tool output, validates the result and flags unsupported or missing details for faculty review. It can generate clinical TXT reports from the brief.
- Stability `stability.stable-image-core-v1:1` supplies the portrait independently. Portrait failure need not invalidate the clinical draft.
- A static example provides an offline demonstration path. Test connection checks actual model access rather than only the presence of a key.

Approach and limits

- Faculty approval is the clinical review boundary. AI output is a draft, not an authoritative diagnosis or automatically published case.
- The current Sonnet 5 integration does not force temperature to zero. Structured validation and reply caching help consistency but do not guarantee identical outputs for every request.
- Uploaded material can be included in generation where the model supports it. This is not a general OCR pipeline or a vector-search knowledge base.
- A generated portrait is not a newly trained 3D avatar; the encounter uses existing avatar assets.

Presentation answer: "We combine model-generated structure with schema checks and faculty review. The case only becomes a student-facing patient after approval."

4 Approval, files and version history

Workflow step 3: publish a stable patient version.

What approval saves

- Approved patient profile JSON and its attachment manifest go to PostgreSQL. The manifest identifies the files belonging to that version.
- PDF/TXT and other patient assets go to the private S3 bucket using versioned/content-based object keys and checksums.
- The live workspace in SQLite records the published case after cloud storage succeeds. Failed approval leaves the draft or previous published version available.
- Existing approved versions remain separate from later edits. Encounters freeze the patient profile, version, rubric and relevant documents when they start.

Tools and implementation

- The storage layer uses PostgreSQL transactions for profile and manifest records and S3 APIs for objects.
- Uploads occur before the PostgreSQL commit. PostgreSQL and S3 do not share an atomic transaction; a failed commit can leave an unreferenced object to reconcile.
- Attachment reads use authenticated application routes. Private S3 storage is not converted into public patient-file links.
- Profile deletion uses preserved encounter/version history so prior work remains reviewable.

Demo recovery and preserved data

- The Aurora gateway accepted TCP but failed TLS before authentication. Reboot and stop/start did not resolve that gateway failure.
- The app now uses a PostgreSQL 16 database on the existing EC2 instance, with password authentication, TLS verification and a listener restricted to loopback.
- All seven source tables were restored from the verified backup and their original row contents checked before reconciliation.
- The recovered deployment preserves nine user accounts and the current workspace: 27 cases, 96 encounters and 47 attachments. Newer local records reconciled the version count to 10 and speech metadata to 35.
- Restored session and OAuth-state rows were invalidated deliberately. Users sign in again; account IDs, password hashes, Google identities and roles were preserved.
- Aurora changes after the last readable backup may be missing. All current encounter owners matched recovered accounts. The original Aurora database remains intact.

Presentation answer: "The demo uses local PostgreSQL on AWS EC2 for durable account and approval records, private S3 for files, and SQLite for encounter operations. We preserved the existing data and validated the real application workflow after migration."

5 Student encounters: chat, voice and video

Workflow step 4: practice with a patient.

Shared encounter behavior

- A student selects a published patient. The encounter saves conversation turns automatically in SQLite and uses its frozen patient version.
- Vitals, findings and documents become available as the student asks relevant questions or meets disclosure/rapport conditions. Faculty-only material is excluded from the patient dialogue context.
- Sonnet 5 produces patient replies using the currently eligible facts. Returned fact IDs, numbers and output length are checked, with a retry and fallback path.
- A reply cache keys responses to patient version, normalized question, disclosure state/rapport and dialogue engine. It is not a guarantee against every model hallucination.
- Three.js and TalkingHead.js display the animated avatar. Amazon Polly speech and timing marks support spoken replies; browser speech synthesis is a fallback.

Voice mode

- Browser Web Speech recognition converts the student's speech into text, which follows the same dialogue and saved-transcript flow as chat.
- Tap-to-speak controls start/stop input. Browser support and microphone permission are required; speech recognition may use the browser provider's service.
- Recording for delivery coaching is a separate, optional opt-in. Voice input does not imply permanent storage of every microphone sample.

Video mode

- HTTPS camera/microphone permission is requested through `getUserMedia`. The muted, mirrored student preview appears in the avatar frame's upper-left corner.
- A square, rounded preview is constrained to at most 8% of the animation-frame area; resize handling keeps it within the frame.
- No video frames, screenshots or encoded video are uploaded or stored. There is no visual emotion, face or gesture analysis.
- For optional coaching, the browser creates an audio-only stream from the microphone tracks. It does not record a video file and then extract audio from it.
- Tracks stop on mode changes, navigation, logout and encounter completion. Late permission responses are guarded against leaking a live stream.

Presentation answer: "Video is a local self-view. Only microphone audio can enter the opted-in coaching pipeline; the video itself never reaches our backend."

6 Audio coaching and communication feedback

Workflow step 5: reflect on delivery, then finish.

Audio capture, processing and storage

- Opted-in speaking turns use AudioWorklet capture to produce 16 kHz, mono, 16-bit WAV audio, up to 90 seconds per turn. A long encounter consists of multiple turns.
- The backend validates consent, encounter ownership, WAV format, size and checksum. A durable SQLite outbox retries storage across restarts/outages.
- Raw clips go to private S3. PostgreSQL stores speech metadata; the workspace records job state. Temporary staging is removed after upload.
- Authenticated playback is limited to the student owner and faculty. Students can delete their recordings early. Playback expires after seven days; background deletion can lag during an AWS outage.
- Deleting/expiring audio does not erase the text transcript or saved measurements. The app does not need to retain recordings forever to keep coaching results.

What the audio analysis actually measures

- Energy measurements estimate speaking spans and quiet gaps; recognized words and estimated spans support pace calculation.
- Delivery feedback includes pace, pauses, audio level/clipping and an approximate playback-overlap signal.
- Microphone gain, background noise, browser recognition and the energy threshold affect these measurements.
- Audio coaching does **not** infer emotion, empathy, personality or labels such as "rude." It does not modify the clinical rubric score.

Text communication signals

- Deterministic phrase-based checks describe warmth as warm, neutral or cold and identify empathy statements, open/closed questions, jargon and other communication signals.
- Evidence links back to conversation turns for reflection. These heuristics are not a validated clinical emotion or empathy instrument.
- Voice/video transcripts use the same text-based checks. Acoustic delivery measurements remain a separate feedback panel.

Presentation answer: "We offer observable delivery coaching, not an audio emotion classifier. Empathy-related wording comes from transcript heuristics. Faculty assessment remains separate from both."

7 Assessment, history and faculty review

Workflow step 6: review evidence and release feedback.

Automated evidence and faculty assessment

- Finishing an encounter prepares deterministic scoring evidence from clinical criteria, keywords and scenario rules. Critical, important and supporting criteria have different weights.
- The model does not assign the final clinical grade. Faculty review the automatic suggestions and conversation evidence.
- Seven domains cover opening, rapport, information gathering, cues, explanation/shared decisions, reasoning/synthesis and professional/safety closure.
- Faculty assign 1–4 or N/A per domain. The maximum is 28 when all seven apply; N/A domains reduce the denominator. An all-N/A review has no numerical score.
- All seven domains require review before release. Low-domain patterns trigger review, not an automatic pass/fail decision.
- Review versions and released snapshots protect previously released feedback from later unreleased edits. Input hashes identify the profile/rubric/conversation being assessed.

History and operational feedback

- Students see their own saved encounters and released assessment. Faculty can review all students' encounters and metrics.
- History includes transcript and case-version context. Student-safe exports omit protected faculty content.
- Student conversation ratings and faculty ratings of automatic-score accuracy feed the Feedback & reports view.
- Report a problem captures relevant page/encounter/turn context. Faculty can resolve or reopen reports; reporting has rate limits.
- Developer exports and diagnostics are protected behind a separate unlock.

A concise live-demo sequence

1. Sign in as faculty, show Studio and generate or load a prepared patient draft.
2. Inspect a disclosure rule and report, then approve the patient.
3. Sign in as a student, start the case, ask a baseline and a more targeted question.
4. Show voice/video controls and explain the optional audio-only coaching boundary.
5. Finish; switch to faculty, inspect evidence and release a complete seven-domain review.
6. Return to the student history and show the released result.

Avoid overclaiming: The app is a synthetic practice prototype. Faculty self-selection, text heuristics and acoustic measurements do not establish clinical validity or replace institutional assessment.

8 Hosting, recovery and team Q&A

Workflow step 7: keep the demonstration reliable.

Tools and deployment approach

- Namecheap DNS points the public domain to EC2. Nginx terminates HTTPS with Let's Encrypt and proxies the Python application managed by systemd.
- GitHub Actions checks backend/frontend behavior and syntax. The EC2 deploy timer follows eligible main revisions, uses versioned releases and health checks, and supports rollback.
- The release includes the connection-pool fix, backup/service source and this guide. Private credentials and backups stay outside Git.
- PostgreSQL now uses loopback-only TLS and a restricted application role. Private credentials remain in root-only EC2 environment files.
- A systemd timer saves PostgreSQL and SQLite recovery snapshots every 15 minutes, retaining 24 snapshots on EC2. This is not an off-site disaster-recovery copy.
- Workshop EC2, S3 and Bedrock resources/credentials still have workshop lifecycle limits. Moving the database removes the Aurora gateway dependency, not those account limits.

What was verified after the migration

- Account signup, login, logout, persisted sessions and concurrent account reads; real Sonnet 5 invocation and S3/PostgreSQL access.
- Faculty approval stores profile/version metadata and a TXT file in S3; student ownership checks reject another student's access.
- Encounter creation, attachment retrieval, turn saving, completion, student/faculty history and seven-domain feedback.
- Audio upload, metadata, exact playback bytes and deletion; Google OAuth start/state and production callback configuration. Interactive Google consent was not re-completed in this check.
- Backend tests: 153 run, one skipped. Frontend render and asynchronous media/privacy checks passed. Synthetic verification records were cleaned up.

Likely questions and consistent answers

- **Is all data in PostgreSQL?** No. Account/approved-version/audio metadata is there; encounters/transcripts and working records remain in SQLite; files/audio are in S3.
- **Does video improve emotion scoring?** No video analysis is implemented. Video provides a local self-view; optional audio provides delivery measurements.
- **Is grading done by AI?** Dialogue/generation use AI. Clinical scoring evidence is deterministic and final released assessment is faculty-reviewed.
- **Are we fully recovered from Aurora?** The demo is running from a verified backup plus current workspace reconciliation. Post-backup Aurora-only changes may be unavailable.
- **Is HTTP health sufficient?** No. Check account access, model connection and a real encounter before presenting.

Implementation evidence for follow-up questions

`backend/auth.py`, `backend/cloud_storage.py`, `backend/generator.py`

`backend/dialogue.py`, `backend/speech.py`, `backend/scoring.py`

`backend/sentiment.py`, `backend/server.py`; frontend modules and deployment scripts. Also see the repository's authentication, speech-delivery, EC2, storage and demo-recovery guides.